

FEU Institute of Technology

COLLEGE OF ENGINEERING · COLLEGE OF COMPUTER STUDIES

Department of Information Technology

CCS0007 – COMPUTER PROGRAMMING 2

AN11 – Term 2

Midterm Machine Problem

Grade

Submitted by:

Hizon, Jed L.

Submitted to

Mr. Mark Anthony Cezar

Submitted on:

Feb. 10, 2024

I. Machine Problem 1

Title: **Student Database Management System**

Problem Statement:

You are tasked with creating a simple Student Database Management System in C++ that allows users to store and manipulate student information. The program should utilize user-defined functions, arrays, character and string manipulation, as well as structures.

Requirements:

Student Structure:

Create a structure named **Student** with the following members:

- int studentID
- char firstName[50]
- char lastName[50]
- float GPA

Function Definitions:

a. void addStudent(Student students[], int &numStudents)

This function should take as parameters an array of Student structures (students) and an integer reference (numStudents) indicating the current number of students in the database.

It should prompt the user to input details for a new student and store them in the array.

b. void displayStudents(const Student students[], int numStudents)

This function should take the array of Student structures and the total number of students as parameters.

Display the details of all students in a formatted tabular manner.

c. float calculateAverageGPA(const Student students[], int numStudents)

This function should take the array of Student structures and the total number of students as parameters.

Calculate and return the average GPA of all students.

d. void findStudentsByLastName(const Student students[], int numStudents, const char lastName[])

This function should take the array of Student structures, the total number of students, and a last name as parameters.

Display the details of students whose last name matches the given input.

Main Program:

In the main program, create an array of Student structures to hold up to 100 students. Initialize the numStudents variable to 0. Implement a menu-driven program with the following options:

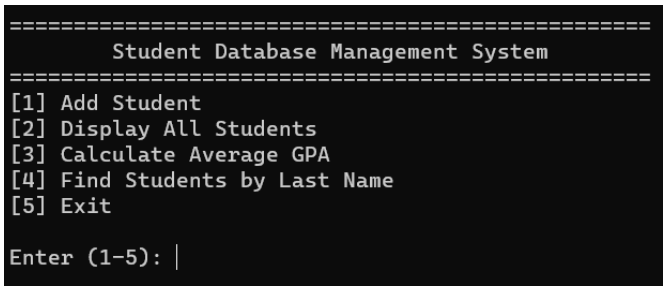
- Add Student
- Display All Students
- Calculate Average GPA
- Find Students by Last Name
- Exit

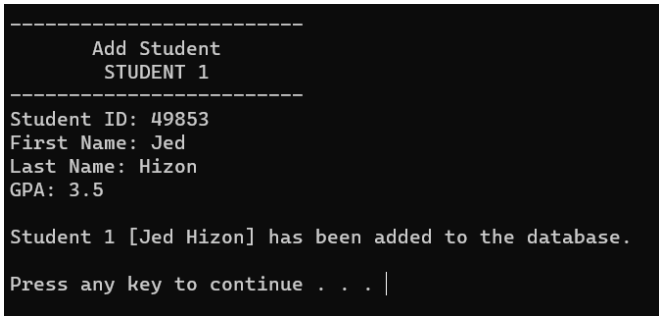
Utilize the functions defined above to perform the corresponding actions for each menu option.

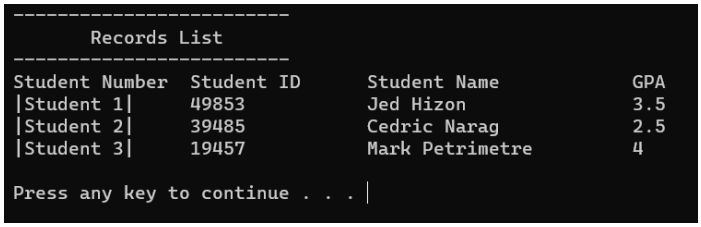
Note: Ensure you perform necessary input validation and handle edge cases gracefully (e.g., when the user tries to add more students than the array can hold).

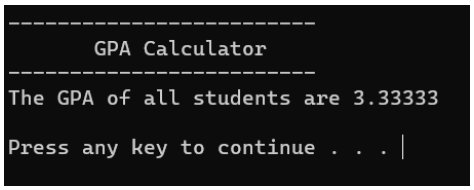
II. Submission

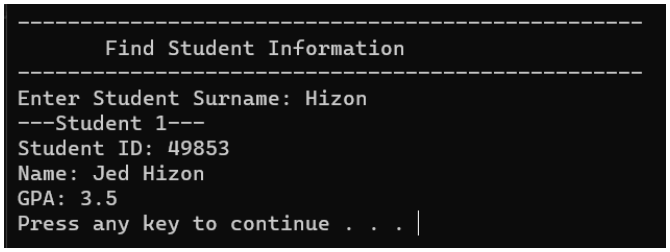
Paste the source code and include 4 screenshots of the program output here.











```
#include <iostream>
#include <string.h>
#include <limits>

using namespace std;

const int longLine = 50;
const int shortLine = 25;

struct Student{
	int studentID;
	char firstName[50];
	char lastName[50];
	char fullName[100];
	float GPA;
};
void end();
void menu();
void error();
void addStudent(Student students[], int &numStudents);
void displayStudents(const Student students[], int numStudents);
float calculateAverageGPA(const Student students[], int numStudents);
void findStudentsByLastName(const Student students[], int numStudents, const char lastName[]);

int main() {
	char choice = 0, lastName[50];
	int numStudents = 0;
	float resultGPA = 0;
	Student students[100];
	bool checkForRecord = false;

	do{
		menu(); //print menu to avoid clutter inside main()
		cout << "Enter (1-5): ";
		cin >> choice;

		switch(choice){
			case '1': //add student record
				if(numStudents < 100){ //checks if the number of students are 100 and below
					addStudent(students, numStudents);
					checkForRecord = true;
					end();
				}
				else{
					error();
					cout << "Maximum student record reached.\n\n";
					end();
				}
				break;
```

```
case '2': //view student record
        if(checkForRecord == true){ //if no record yet, output error
                displayStudents(students, numStudents);
                cout << endl;
                end();
        }
        else{
                error();
                cout << "Please choose an option (1-5).\n\n";
                end();
        }
        break;
case '3': //calculate total gpa
        if(checkForRecord == true){
                system("cls");
                cout << string(shortLine, '-') << endl;
                cout << "       GPA Calculator\n";
                cout << string(shortLine, '-') << endl;

                resultGPA = calculateAverageGPA(students, numStudents);
                cout << "The GPA of all students are " << resultGPA << endl << endl;
                end();
        }
        else{
                error();
                cout << "Please choose an option (1-5).\n\n";
                end();
        }
        break;
case '4': //search last name
        if(checkForRecord == true){
                system("cls");
                cout << string(longLine, '-') << endl;
                cout << "       Find Student Information\n";
                cout << string(longLine, '-') << endl;

                cout << "Enter Student Surname: ";
                cin >> lastName;

                findStudentsByLastName(students, numStudents, lastName);
                end();
        }
        else{
                error();
                cout << "Please choose an option (1-5).\n\n";
                end();
        }
        break;
```

```
                                case '5': //exit program
                                        system("cls");
                                        cout << "Program Exited." << endl;
                                        return 0;
                                default:
                                        error();
                                        break;
                        }
                }
                while(choice != '5');
                return 0;
}

void end(){ //shortened for convenience
                system("pause");
                system("cls");
}
void error(){ //shortened for convenience
                system("cls");
                cout << string(shortLine, '*') << endl;
                cout << "        ERROR!\n";
                cout << string(shortLine, '*') << endl;
                cout << "Invalid Action!\n";
}
void menu(){
                cout << string(longLine, '=') << endl;
                cout << "\tStudent Database Management System\n";
                cout << string(longLine, '=') << endl;

                cout << "[1] Add Student\n";
                cout << "[2] Display All Students\n";
                cout << "[3] Calculate Average GPA\n";
                cout << "[4] Find Students by Last Name\n";
                cout << "[5] Exit\n\n";
}
```

```
void addStudent(Student students[], int &numStudents){
	system("cls");

	bool isNum = true, isGPA = true;
  do {
	isNum = true;
	cout << string(shortLine, '-') << endl;
		cout << "	Add Student\n";
		cout << "	 STUDENT " << numStudents+1 << endl;
		cout << string(shortLine, '-') << endl;

    cout << "Student ID: ";
    cin >> students[numStudents].studentID;

    if(cin.fail()) { //print error if input for studentID is a non-digit
		system("cls");
		error();
      cout << "Please input numbers only.\n\n";

      isNum = false;
      cin.clear();
      cin.ignore(numeric_limits<streamsize>::max(), '\n');
      end();
    }
  } while (isNum == false);

  cout << "First Name: ";
  cin >> students[numStudents].firstName;
  cout << "Last Name: ";
  cin >> students[numStudents].lastName;

	do{
	cout << "GPA: ";
	isGPA = true;
	cin >> students[numStudents].GPA;

	//checks if gpa is a 0-4 digit, otherwise output error and let user input another
	if(cin.fail() || students[numStudents].GPA < 0 || students[numStudents].GPA > 4){
		cout << "\nPlease input proper GPA (0.0 - 4.0).\n\n";
		cin.clear();
		cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
		isGPA = false;
		}
	} while(isGPA == false);

  cout << "\nStudent " << numStudents + 1 << " [" << students[numStudents].firstName << " " <<
students[numStudents].lastName << "] has been added to the database.\n\n";
  numStudents++; //increase the amount of students on record
}
```

```
void displayStudents(const Student students[], int numStudents){
	system("cls");
	cout << string(shortLine, '-') << endl;
	cout << "	Records List\n";
	cout << string(shortLine, '-') << endl;

	cout << "Student Number\t" << "Student ID\t" << "Student Name\t\t" << "GPA\n";

	for(int i = 0 ; i < numStudents; i++){ //print all of the recorded students
		cout << "|Student " << i+1 << "|\t";
		cout << students[i].studentID << "\t\t";
		cout << students[i].firstName << " " << students[i].lastName; cout << "\t\t";
		cout << students[i].GPA << endl;
	}
}

float calculateAverageGPA(const Student students[], int numStudents){
	double totalGPA = 0;
	float result = 0;

	for(int i = 0; i < numStudents; i++){
		totalGPA += students[i].GPA; //get the summation of all GPA
	}
	return totalGPA / numStudents;
}

void findStudentsByLastName(const Student students[], int numStudents, const char lastName[]){
	int result = 0;
	bool foundStudent = false;

	for(int i = 0; i < numStudents; i++){
		result = strcmp(lastName, students[i].lastName); //compares if lastName is the same

		if(result == 0){ //if the comparison is the same
			cout << "---Student " << i+1 << "---" << endl;
			cout << "Student ID: " << students[i].studentID << endl;
			cout << "Name: " << students[i].firstName << " " << students[i].lastName << endl;
			cout << "GPA: " << students[i].GPA << endl;
			foundStudent = true;
		}
	}
	if(foundStudent == false){
		cout << "No student record found.\n";
	}
}
```

III. Rubrics

TRAIT	Exceptional	Acceptable	Amateur	Unsatisfactory
Specifications (20)	The program works and meets all of the specifications.	The program works and produces the correct results and displays them correctly. It also meets most of the other specifications.	The program produces correct results but does not display them correctly.	The program is producing incorrect results.
Readability (20)	The code is exceptionally well organized and very easy to follow.	The code is fairly easy to read.	The code is readable only by someone who knows what it is supposed to be doing.	The code is poorly organized and very difficult to read.
Reusability (20)	The code could be reused as a whole or each routine could be reused.	Most of the code could be reused in other programs.	Some parts of the code could be reused in other programs.	The code is not organized for reusability.
Documentation (30)	The documentation is well written and clearly explains what the code is accomplishing and how.	The documentation consists of embedded comment and some simple header documentation that is somewhat useful in understanding the code.	The documentation is simply comments embedded in the code with some simple header comments separating routines.	The documentation is simply comments embedded in the code and does not help the reader understand the code.
Efficiency (10)	The code is extremely efficient without sacrificing readability and understanding.	The code is fairly efficient without sacrificing readability and understanding.	The code is brute force and unnecessarily long.	The code is huge and appears to be patched together.